

- RAPPRESENTAZIONE DI UN GRAFO IN MEMORIA:

- 2 metodi
- * LISTE DI ADIACENZA
- * MATRICE DI ADIACENZA

- **LISTE DI ADIACENZA**: rappresentazioni compatte per grafi **SPARTI** $\Rightarrow (|E| \ll |V|^2)$, con $E = \text{"archi"}$, $V = \text{"vertici"}$
 * Costituite da un **Array** di **dimensione** $|V|$ di liste dinamiche.
 * Ogni casella i contiene la lista dei vertici collegati a i .
 ossia "moltissimi vertici poco collegati".

- SPAZIO RICHiesto:

- * Grafo orientato $\rightarrow |E| \rightarrow \sum$ lunghezze di tutte le liste.
- * Grafo non " " $\rightarrow 2|E| \sim$ (Informazioni compaiono 2 volte)
 $\downarrow$ su ogni vertice.

In generale, orientato o meno: $O(|V| + |E|)$

Svantaggio nell'uso $\rightarrow$ TEMPI DI ESECUZIONE (della ricerca): $O(|V|)$

$\downarrow$
 è proprio a risolvere le tempistiche che si usa la M.

- MATRICE DI ADIACENZA: (grafo generico non pesato)

- * Vertici numerati da 1 a $|V|$
 - * Matrice $|V| \times |V| \leftarrow$ **booleani** $\begin{cases} 0: \text{"non collegati"} \\ 1: \text{"collegati"} \end{cases}$
 - * Spazio: $\Theta(|V|^2)$
- $A[i,j] = \begin{cases} 1 & \text{se } (i,j) \in E. \\ 0 & \text{Altrimenti.} \end{cases}$

! Se il grafo è **NON ORIENTATO**, avrà una matrice simmetrica.

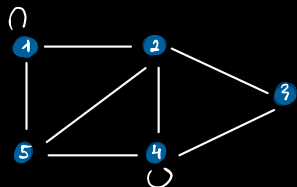
$\rightarrow$ Nel caso io sappia questa cosa a priori $\rightarrow$ mi salvo solo i valori (sopra/sotto) la metà diagonale (RISPARMIO 50%).

! Se il grafo è **PESATO**, $\Rightarrow$ Matrice diventa di interi.

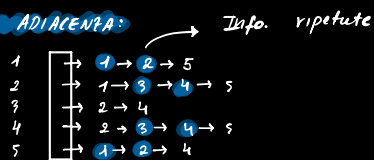
$\rightarrow$ Nel caso 2 vertici non siano collegati si mette ∞ .

Vantaggio: a sapere se 2 V sono collegati $\Rightarrow$ **SEMPLICE CONSULTA DI MATRICE**.

Esempi:



L. ADIACENZA:



MATRICE ADIACENZA

	1	2	3	4	5
1	1	1	0	0	1
2	1	1	0	1	1
3	0	1	1	1	1
4	0	1	1	1	1
5	1	1	1	1	1

$\rightarrow$ tutto questo me lo risparmio

- ALGORITMI DI VISITA DEI GRAFI:

- * **BFS**: Breadth (AMPIEZZA)
- * **DFS**: Depth (PROFONDITÀ)

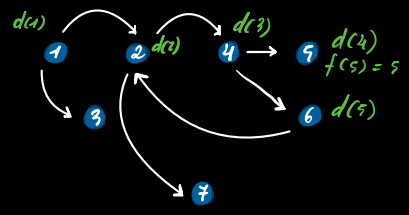
- ALGORITMO DFS (BFS con Dennonno)

Consideriamo di avere $G(V,E)$. $\Rightarrow$ Depth First Search, esplora il grafo andando sempre il più possibile in profondità, prima di "PROVARE ALTRE STRADE"

- * **Esplora tutti i vertici** (BFS visita solo le componenti connesse invece).
 - per ogni $v \in V$
 - visita il sottografo in profondità.

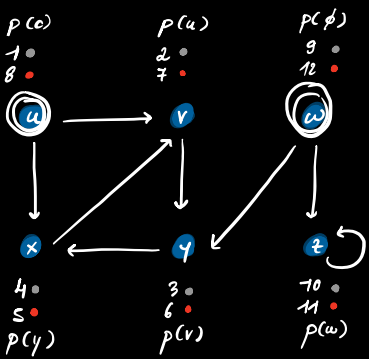
- VISITA DI LISTE DI ADIACENZA:

- * Assegno ad ogni v . un **color[u]**:
 - **bianco-white**: NON SCOPERTO
 - **grigio-gray**: VERTICE SCOPERTO, MA NON ANALIZZATO COMPLETAMENTE.
 - **nero-black**: VERTICE VISITATO COMPLETAMENTE.



- * **Pr[u]**: predecessore (u) nella visita DFS.
- * **d[u]**: tempo in cui viene scoperto il vertice u .
- * **f[u]**: tempo in cui finisce la visita in u . } ovviamente $d(u) < f(u)$ SEMPRE.

- ESEMPIO ESPL:



punti grigi/rossi, sono anche "tempo scoperta, fine scoperta".

P = u v y x ~~x~~ ~~x~~ ~~x~~ ~~x~~ ~~w~~ z ~~w~~ ~~w~~

t = 1 2 3 4 5 6 7 8 9 10 11 12

c = .

"facciamo finta sia nero"

- ALGORITMO:

L'algoritmo della DFS è ricorsivo.

esplorazione della singola component.

DFS(G) $\rightarrow$ lista

Θ(V) For ($v \in V$): {
 color(v) = W
 p(v) = NIL
Time = 0

Θ(V) For (all $v \in V$): {
 if (color(v) == "white") {

SETTING INIZIALE

RICERCA
DEI

DFS-VISIT(G, u) Me ho scoperto un v. bianco

time++
d[u] = time;
color[u] = G;

nodo "init"
lista di u.

For all $w \in \text{Adj}[u]$:

if (color[w] = W) {
 p[w] = u;

* questo For scorre tutte le liste di adiacenza di ogni nodo della componente corrente.

ESPLORAZIONE IN RICERCA
DEI "BIANCHI ADIACENTI"

DFS-VISIT (v)

BIANCHI

DFS-VISIT (G, w);

RICORDARE (esplorat. prof.)

mi assicura di esplorare tutte le componenti.

color[u] = **B** → Black
time++;
F[u] = time;

el. tot. lista Adj = $|E|$

tempo totale DFS: $\Theta(|V| + |E|)$

TEST per capire se un nodo fa parte del sottografo di esplorazione di un altro nodo, guardando solamente i tempi.

TEOREMA DELLE PARENTENI: 2 nodi di un grafo possono trovarsi solo in 3 sit. tra loro.

* DFS di $(G, V, E) \Rightarrow \forall u, v \in V; A = [d[u], f[u]] \quad B = [d[v], f[v]]$ prende il nome da QUESTE parenteni.

- $A \cap B = \emptyset \rightarrow$ non sono in relaz. tra loro.
- $A \subseteq B \rightarrow$ prima scopro B, A sta in un sottografo di B.
- $B \subseteq A \rightarrow$ prima scopro A, B sta in un sottografo di A.

"SOTTOGRAFO"

Questo risponde alla domanda "vale la pena esplorare (a) → (b)?".

TEOREMA DEL CAMMINO BIANCO:

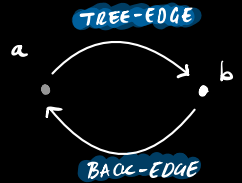
Se mi trovo nel 2° e 3° caso:

A → B: qui in mezzo, da A a B si trovano "n" nodi bianchi da esplorare. Vale uguale da B → A.

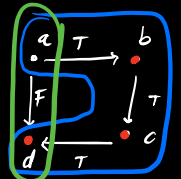
$n \geq 0$ (magari sono dirett. colleg.)

Classificazione di Archi:

- **U** → Tree-edge: arco ∈ Alla "Foresta DFS" (albero di scoperta)
* lati che portano a scoprire nuovi vertici. → "trovo un bianco"
- **G** → Back-edge: arco ∉ alla "Foresta DFS"
* Vanno da un vertice v ad un antenato di v nell'albero DFS. → "trovo un grigio"



- **B** → Forward-edge: è un arco che ancora non avevo percorso, ma che mi porta ad un sottografo che avevo già esplorato.
è un cammino alternativo per fare un'esplorazione che ho già eseguita.



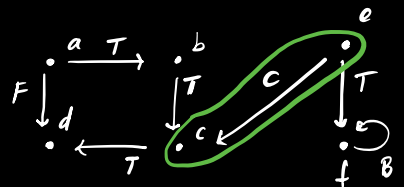
prima esploraz.
seconda expl.

Ordina il grafo → d è un "figlio di" a. Sta nel sottografo di a.

- **B** → Cross-edge:

È un arco verso un nodo nero, CHE COLLEGA 2 COMPONENTI DIFFERENTI.
Come distinguo il F-E dal C-E?

Con il teorema delle parenteni → ottengo il caso $A \cap B = \emptyset$
DISTINZIONE CROSS-FORWARD!

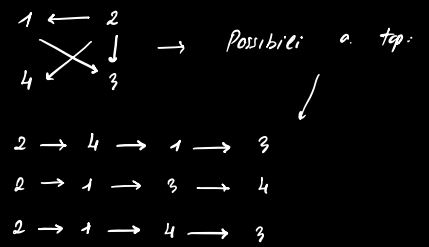


ESEMPIO APPLICAZIONE: ORDINAMENTO TOPOLOGICO

Input: DAG (Grafo diretto aciclico)

Output: Lista ordinata e lineare dei vertici di G . t.c.

Se G contiene l'arco (u, v) Nell'ordinamento u precede v .



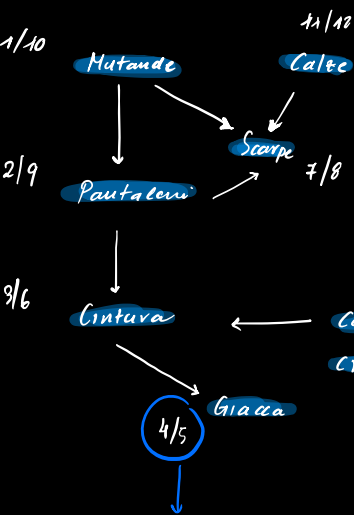
Topologic-sort (G)

DFS (G) /* fine visita $\forall v \in V$ $f[v] \neq \perp$ /

Ordina decrescente per $f[v]$

Output (Ordine).

*Esempio: (topologic sort)



Topologia:

Camicia
Cravatta
Calze
Mutande
Pantaloni
Scarpe
Cintura
Giacca
Orologio

$i = \text{tempo scoperta} / j = \text{tempo fine esplorazione}$

*Piccoli "ragionamenti":

Algoritmo DFS:

- 1) Aggiungo informazioni in più ad ogni nodo del G .
 - colore (bianco, grigio, nero).
 - predecessore
 - tempi di scoperta
- 2) L'algoritmo è "spezzato in 2":

DFS $\rightarrow$ si assicura di lanciare DFS-VISIT per ogni componente.

DFS VISIT $\rightarrow$ esplora la singola componente.

3) "COLTELINO SVIZZERO DI CONOSCENZA"

$a \in \text{ad un sottografo di } b?$
 $b \in \text{" di } a?$ $\rightarrow$ uso il teorema delle parentesi.

4) DISTINZIONE DEGLI ARCHI $\rightarrow$ derivata.

TABELLA CONTENUTI: (20/11)

